

工程化思维和框架必要性—复习要点

1 项目现状分析与工程化重构

1.1 问题一：DOM 操作与业务逻辑耦合

[p.7–13]

- > 问题表现 [p.7]: HTML 结构直接嵌入 JavaScript 字符串中，模板与逻辑混杂
- > 四方面问题 [p.8]:
 - 违反关注点分离原则 (Separation of Concerns): UI 结构嵌在 JS 中，前端设计师难以参与
 - 代码可读性与可维护性差: 字符串拼接难以阅读，缺乏语法高亮
 - 潜在安全风险: 直接拼接服务端数据存在 **XSS** 风险
 - 难以单元测试: 渲染逻辑与 DOM 操作耦合，测试需要真实 DOM 环境
- > 解决方案 [p.9–13]:
 - 方案一: 封装模板类 [p.9–10]
 - 方案二: 使用模板引擎 (Template Engine) 更优雅地分离 HTML 结构和 JS 逻辑 [p.11–13]

1.2 问题二：重复代码与代码复用性差

[p.14–17]

- > 问题 [p.15]: 两个函数都执行”清空容器 → 遍历数据 → 生成 HTML → 插入 DOM”，相同模式重复出现；修改渲染逻辑需在多处修改
- > 解决方案 [p.16–17]: 封装统一渲染工具类，提取公共渲染逻辑

1.3 问题三：错误处理机制不完善

[p.18–21]

- > 问题 [p.19]: 只捕获异常并抛出，无友好错误提示；缺乏错误分类（网络/服务器/超时）；没有重试机制
- > 解决方案 [p.20–21]: 构建统一异常处理类，区分错误类型、提供友好提示、实现重试

1.4 问题四：缺乏真正的模块化与组件化

[p.22–24]

- > 问题 [p.23]:
 - 静态 HTML 包含的局限性: 只能包含静态内容，无法传递参数
 - 组件间通信困难: 依赖全局变量或 DOM 操作
 - 缺乏生命周期管理: 没有挂载、更新、销毁等生命周期钩子
 - 难以进行单元测试
- > 解决方案 [p.24]: 构建组件系统，支持参数传递、生命周期、组件间通信

1.5 重构小结

[p.25–27]

- > 通过四个问题的解决，从原始的 DOM 操作代码逐步重构为具有模板引擎、统一渲染、错误处理和组件化能力的工程化项目 [p.25–27]

2 前端架构模式

2.1 MVC 架构模式

[p.29–33]

- > **MVC** (Model-View-Controller) [p.29–31]:
 - **Model:** 管理数据和业务逻辑
 - **View:** 负责 UI 展示
 - **Controller:** 处理用户输入, 协调 Model 和 View
- > **优点** [p.31]: 职责分离、Model 可被多个 View 复用、各层可独立测试、灵活修改单层
- > **缺点** [p.31–33]:
 - Controller 随应用复杂化而臃肿
 - View 与 Controller 耦合: View 依赖特定 Controller
 - 数据流不够清晰, 复杂应用数据流向难以追踪

2.2 MVP 架构模式

[p.34–39]

- > **MVP** (Model-View-Presenter) [p.34]: MVC 的改进版本, 主要改进 View 和 Controller 之间的耦合问题
- > **改进点** [p.39]:
 - **View 完全解耦:** 只负责展示, 不包含任何业务逻辑
 - **Presenter 集中处理:** 所有业务逻辑集中在 Presenter 中
 - **更高可测试性:** Presenter 不依赖具体 View 实现, 可独立测试
 - **代码结构更清晰:** 职责划分更明确
- > **对比 MVC:** Controller 被 Presenter 替代, View 通过接口与 Presenter 交互而非直接依赖

2.3 MVVM 架构模式

[p.40–45]

- > **MVVM** (Model-View-ViewModel) [p.40]: 引入 ViewModel 作为 View 和 Model 之间的桥梁
- > **核心特性: 响应式数据绑定** (Reactive Data Binding) [p.42–45]
 - ViewModel 中的数据变化自动反映到 View
 - View 中的用户输入自动更新 ViewModel
 - 实现数据的**双向绑定** (Two-way Data Binding)
- > **简化版响应式系统原理** [p.44–45]: 通过数据劫持/代理监听数据变化, 自动通知 View 更新
- > **代表框架:** Vue、React (单向数据流 + 虚拟 DOM)

3 三大架构模式对比

特性	MVC [p.29-33]	MVP [p.34-39]	MVVM [p.40-45]
核心组件	Model-View-Controller	Model-View-Presenter	Model-View-ViewModel
View 与逻辑	有耦合	完全解耦	通过数据绑定解耦
数据流向	双向，可能混乱	Presenter 居中调度	双向绑定，自动同步
可测试性	中	高	高
代表框架	Backbone.js、早期 Angular	Android 开发	Vue、React、Angular
主要问题	Controller 臃肿	Presenter 代码量大	学习曲线，调试复杂

4 本章小结与考点梳理

[p.46]

- > 工程化分析 [p.7-27]：四大问题（DOM 耦合/代码重复/错误处理/组件化）及其解决方案
- > 前端架构模式 [p.29-45]：MVC（职责分离但 View-Controller 耦合）→ MVP（View 完全解耦，Presenter 集中处理）→ MVVM（响应式数据绑定，双向绑定）

高频考点：(1) 工程化重构中的四大问题及解决方案 [p.7-24]；(2) MVC/MVP/MVVM 三种架构模式的核心思想和演进逻辑 [p.29-45]；(3) MVC 的缺点（Controller 臃肿、View-Controller 耦合）[p.31]；(4) MVVM 的响应式数据绑定和双向绑定原理 [p.42-45]；(5) MVP 相比 MVC 的核心改进（View 完全解耦）[p.39]